# List Data Structures

In [1]:
```
l = []
l
```

Out[1]:
```
[]
```

In [4]:
```
l.append(10)
l.append(10)
l.append(10)
l
```

Out[4]:
```
[10, 10, 10]
```

In [5]:
```
l.append(20)
l.append(30)
l
```

Out[5]:
```
[10, 10, 10, 20, 30]
```

In [6]:
```
type(l)
```

Out[6]:
```
list
```

In [7]:
```
l1 = [10 , '2.3', 1+2j, True, 'krith', [1,2,3,4]]
l1
```

Out[7]:
```
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

## user can define multiple data types --> that's why is home genious & hetorogenious allowed

In [8]:
```
len(l1)
```

Out[8]:
```
6
```

In [9]:
```
print(l)
print(l1)
```

```
[10, 10, 10, 20, 30]
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [11]:
```
print(id(l))
print(id(l1))
```

```
2936752077696
2936799685568
```

In [12]:
```
l2 = [10,20,30,40,50,10]
l2
```

Out[12]:
```
[10, 20, 30, 40, 50, 10]
```

In [13]:
```
len(l2)
```

Out[13]:
```
6
```

In [14]:
```
l2.clear()
```

In [15]:
```
len(l2)
```

Out[15]:
```
0
```

In [16]:
```
l2
```

Out[16]:
```
[]
```

In [17]:
```
del l2
l2
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[17], line 2
      1 del l2
----> 2 l2

NameError: name 'l2' is not defined
```

In [18]:
```
l.count(20)
```

Out[18]:
```
1
```

In [19]:
```
l.count(10)
```

Out[19]:
```
3
```

In [20]:
```
l.count(100)
```

Out[20]:
```
0
```

In [22]:
```
a = [1,2,3]
b = [4,5,6]
a.extend(b)
a
```

Out[22]:
```
[1, 2, 3, 4, 5, 6]
```

In [23]:
```
l2 = []
l2
```

Out[23]: []

In [25]:
```
l2.extend(l1)
l2
```

Out[25]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [26]:
```
print(l)
print(l1)
print(l2)
```

```
[10, 10, 10, 20, 30]
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
[10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [27]:
```
l[0]
```

Out[27]: 10

In [ ]:
```
# list is mutable
```

In [28]:
```
l[0] = 100
l
```

Out[28]: [100, 10, 10, 20, 30]

In [30]:
```
job = [5000]
job[0]
```

Out[30]: 5000

In [31]:
```
job[0] = 20000
job
```

Out[31]: [20000]

In [32]:
```
l[3] = 25
l[3]
```

Out[32]: 25

In [33]:
```
l
```

Out[33]: [100, 10, 10, 25, 30]

In [34]:
```
l[-2] = 600
l
```

Out[34]: [100, 10, 10, 600, 30]

In [35]:
```
l.index(600)
```

Out[35]: 3

In [36]:
```
l2
```

Out[36]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [38]:
```
l2.index(10)
```

Out[38]: 0

In [39]:
```
l
```

Out[39]: [100, 10, 10, 600, 30]

In [40]:
```
l.insert(2,25)
l
```

Out[40]: [100, 10, 25, 10, 600, 30]

In [41]:
```
l
```

Out[41]: [100, 10, 25, 10, 600, 30]

In [42]:
```
l2
```

Out[42]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [43]:
```
l2.pop()
```

Out[43]: [1, 2, 3, 4]

In [ ]:
```
# pop function will remove last element by default
```

In [44]:
```
l2.pop()
```

Out[44]: 'krith'

In [45]:
```
l2
```

Out[45]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True]

In [46]:
```
l2.pop(2)
```

Out[46]: 10

In [47]:
```
l2
```

Out[47]: [10, 10, 20, 30, 10, '2.3', (1+2j), True]

In [ ]:
```
# pop(2) -- 2nd index value will be eleminated
# remove == remove will removed by value
```

```
# pop vs remove --- pop is removed by index and remove is removed by value.
# by default pop will remove last element if we didn't mention any argument
```

In [48]:
```
l2.remove(30)
```

In [49]:
```
l2
```

Out[49]:
```
[10, 10, 20, 10, '2.3', (1+2j), True]
```

In [50]:
```
l2.reverse()
l2
```

Out[50]:
```
[True, (1+2j), '2.3', 10, 20, 10, 10]
```

In [51]:
```
l
```

Out[51]:
```
[100, 10, 25, 10, 600, 30]
```

In [52]:
```
l.sort()
```

In [53]:
```
l
```

Out[53]:
```
[10, 10, 25, 30, 100, 600]
```

In [54]:
```
l.sort(reverse=True)
```

In [55]:
```
l
```

Out[55]:
```
[600, 100, 30, 25, 10, 10]
```

In [56]:
```
l3 = [10,20,[20,30],30]
l3
```

Out[56]:
```
[10, 20, [20, 30], 30]
```

In [60]:
```
l3[2] = [20]
l3
```

Out[60]:
```
[10, 20, [20], 30]
```

In [ ]: